

國立交通大學

資訊科學與工程研究所

碩 士 論 文

Strength Adjustment and Assessment for MCTS-Based Programs

研究生：劉安仁

指導教授：吳毅成 教授

中 華 民 國 108 年 07 月

Strength Adjustment and Assessment for MCTS-Based Programs

研究生：劉安仁

指導教授：吳毅成

國立交通大學 資訊科學與工程研究所

摘要

這篇論文提出了蒙地卡羅遊戲程式調整強度及評估強度的方法。我們使用強度索引修改了現有的 softmax 策略進行決策。最重要的變動是透過排除較少模擬次數的選步以篩選掉品質較差的決策，而篩選次數由選步中最大模擬次數之預設篩選比率決定。藉由理論分析，我們展示了使用篩選比率可保障選步超越一定的最低強度。此方法應用在圍棋程式 ELF OpenGo 及 CGI。實驗結果顯示出強度索引與實驗強度呈高度正相關。以 0.1 的篩選比率為例，強度索引與 Elo rating 呈線性相關並只有 47.95 Elo 的回歸誤差。在介於 ± 2 的索引值中，此方法可涵蓋約 800 Elo rating 的強度。此外，我們展現了三個動態強度調整並評估對手強度的方法。強度調整及評估也施行在真實世界場景，強度範圍從職業選手（最強）至級位業餘玩家（最弱）。除了提供廣大的強度範圍，我們的方法在相同的強度索引下可穩定地下出相同強度，使得玩家可輕易地選擇合適的對手。強度評估則可在 15 場遊戲內自動地調整棋強度至對手強度，並同時估測出對手強度。就我們所之，此結果為現今強度調整的最佳實踐，因其可涵蓋較廣的 Elo rating 並可維持可控制的強度關係。

Strength Adjustment and Assessment for MCTS-Based Programs

Student : An-Jen Liu

Advisor : I-Chen Wu

Institute of Computer Science and Engineering

College of Computer Science

National Chiao Tung University

Abstract

This paper proposes an approach to strength adjustment and assessment for MCTS-based game-playing programs. We modify an existing softmax policy with a strength index to choose moves. The most important modification is a mechanism which filters low quality moves by excluding those that have a lower simulation count than a pre-defined threshold ratio of the maximum simulation count. Through theoretical analysis, we show that the adjusted policy is guaranteed to choose moves exceeding a lower bound in strength by using a threshold ratio. The approach is applied to the Go programs ELF OpenGo and CGI. Experiment results show that the strength index is highly correlated to the empirical strength; namely, given a threshold ratio 0.1 as an example, the strength index is linearly related to the Elo rating with regression error 47.95 Elo. With an index value between ± 2 , we can cover a strength range of about 800 Elo ratings. We then present three methods to adjust strength and assess opponents' strengths dynamically. The strength adjustment and assessment methods were also tested in real-world scenarios with human players, ranging from professionals (strongest) to kyu rank amateurs (weakest). In addition to offering a wide range of strengths, our method can consistently play at the same strength given the same strength index, allowing players to choose appropriate opponents with ease. In regards to assessment, our method can automatically adjust its strength to its opponent's and in turn measure the opponent's strength accurately within 15 games. To our knowledge, this result is state-of-the-art in terms of the range of strengths in Elo rating while maintaining a controllable relationship between the strength and a strength index.

Table of Content

摘要	i
Abstract	ii
Table of Content	iii
Figures List	v
Tables List	vi
I . Motivation	1
II . Static Strength Adjustment (SSA)	4
2.1 Past Work	4
2.2 Our Approach	5
2.3 Empirical Evaluation	6
III . Theoretical Analysis	9
3.1 Generalized Bradley-Terry Model	9
3.2 Hypothesis	10
IV . Dynamic Strength Adjustment (DSA)	16
4.1 Inter-game Strength Adjustment	16
4.2 Intra-game Strength Adjustment	18
4.3 Mixed Dynamic Strength Adjustment (MDSA)	19
V . Real-World Experiments	21
5.1 Static Strength Adjustment	21
5.2 Dynamic Strength Adjustment	28
VI . Discussion and Future Work	31
6.1 Discussion	31
6.2 Future Work	31

VII. Conclusion	32
Reference	34



Figures List

1	The correlation between z and Elo rating with $R_{th} = 0.1$	7
2	Elo rating (relative to ELF) in different threshold ratios and strength indices. . .	7
3	The number of candidate moves with respect to R_{th}	8
4	The curve of $E_{norm}^{D_{0.1}}(z)$ and the empirical data.	15
5	Strength index estimation for inter-game SA.	17
6	Strength index estimation for MDSA.	20
7	SSA HaiFong professional player rating estimation using WHR (see Table 8 for player name references).	23
8	Dan rank strength correlation to z for the online Go platforms Tygem and FoxWQ. 0 dan corresponds to 1 kyu.	25
9	Tygem dan rank chart over a period of 100 games.	26
10	Kyu rank strength correlation of z on Tygem and FoxWQ. 0 kyu corresponds to 1 dan.	27
11	FoxWQ kyu rank chart over a period of 100 games.	27
12	MDSA strength estimation of selected players.	28
13	MDSA strength estimation of HaiFong professional players.	29

Tables List

1	The win rates (against ELF with $z = 0$) and Elo ratings (relative to the original ELF) with respect to z when $R_{th} = 0.1$	6
2	The conjectured strength indices estimated in different data sets.	15
3	Win rate (WR) and average dynamic strength index (Avg. z) against different opponents using inter-game SA.	17
4	Win rate (WR), average z (Avg. z) and standard deviation of z (Std. z) against different opponents using intra-game SA.	19
5	Win Rate (WR) and average dynamic strength index (Avg. z) against different opponents using MDSA.	20
6	Normalized Contingency Table (row).	24
7	Normalized Contingency Table (column).	24
8	List of 17 HaiFong Professional Players.	26
9	Intra-game SA estimation.	30
10	Comments of P1 (HSU, HAO-HUNG).	30

I. Motivation

Super-human level game playing programs capture the imagination and fascination of society at large; for human players, these programs pose an interesting challenge and offer opportunities for learning. Since the super-human programs AlphaGo Silver et al. (2016) and AlphaGo Zero Silver et al. (2017b), many other programs such as FineArt Tencent AI Lab (2018), Leela Zero Pascutto (2018), and ELF OpenGo Yuandong Tian et al. (2019) have successfully reproduced the AlphaGo Zero algorithm. In addition to Go, AlphaGo Zero’s method was also applied to other games such as chess and shogi, reaching strength levels much higher than human champions Silver et al. (2017a) and state-of-the-art game-playing programs Romstad et al. (2019) Computer Shogi Association (2019).

However, it is also important to fit program difficulty to appropriate levels for purposes of entertainment and education. On the low end of the difficulty scale, human players tend to lose interest when playing against an AI opponent far weaker than themselves; on the other hand, excessive difficulty tends to lead to frustration Hunicke and Chapman (2004). In the context of learning to play with programs, it is difficult to offer feedback for human players if they consistently win or lose. Thus, in order to achieve an overall better game experience, and to improve the learning process for players, it is imperative to balance program difficulty accordingly.

In addition to matching the player with an appropriate opponent, strength adjustment can also be used as a player assessment tool. More specifically, given a player of unknown strength, an equally matched opponent (i.e. playing at about 50% win rate) can be used as a metric of measuring the player’s performance Demediuk et al. (2017). Ideally, the goal is to design a mechanism that is able to provide a wide ranging and evenly distributed set of playing strengths, such that it would be possible for most human players to choose the most appropriate strength

to play against. Similarly, any player that falls within the range of provided strengths can also be measured by this mechanism.

Coming up with a stable and wide-ranging strength adjustment mechanism is a non-trivial problem. Using the AlphaGo Zero algorithm as an example target, there are two straightforward tweaks that can be applied to the two major components of the algorithm: Monte-Carlo tree search (MCTS) and deep neural networks (DNNs).

First, we could reduce the super-human program strength to lower levels by reducing the total thinking time, or the total simulation count in MCTS. However, with this method, the search tree’s relatively smaller size leaves the program vulnerable to tactical traps. For example, the ladder problem in Go is one of the most elementary shapes taught to human players Yuandong Tian et al. (2019); however, for MCTS-based programs, a significant number of simulations is often required before the program can handle ladders properly. It has been shown that when adjusting program strength through reduction, simulation count and playing strength do not form a linear relationship Sephton et al. (2015). In fact, once the number of simulations fall below a certain threshold, the program playing strength drops catastrophically. In other words, by reducing the simulation count, we would not be able to offer an evenly distributed skill level, especially towards the lower end of the playing strength scale.

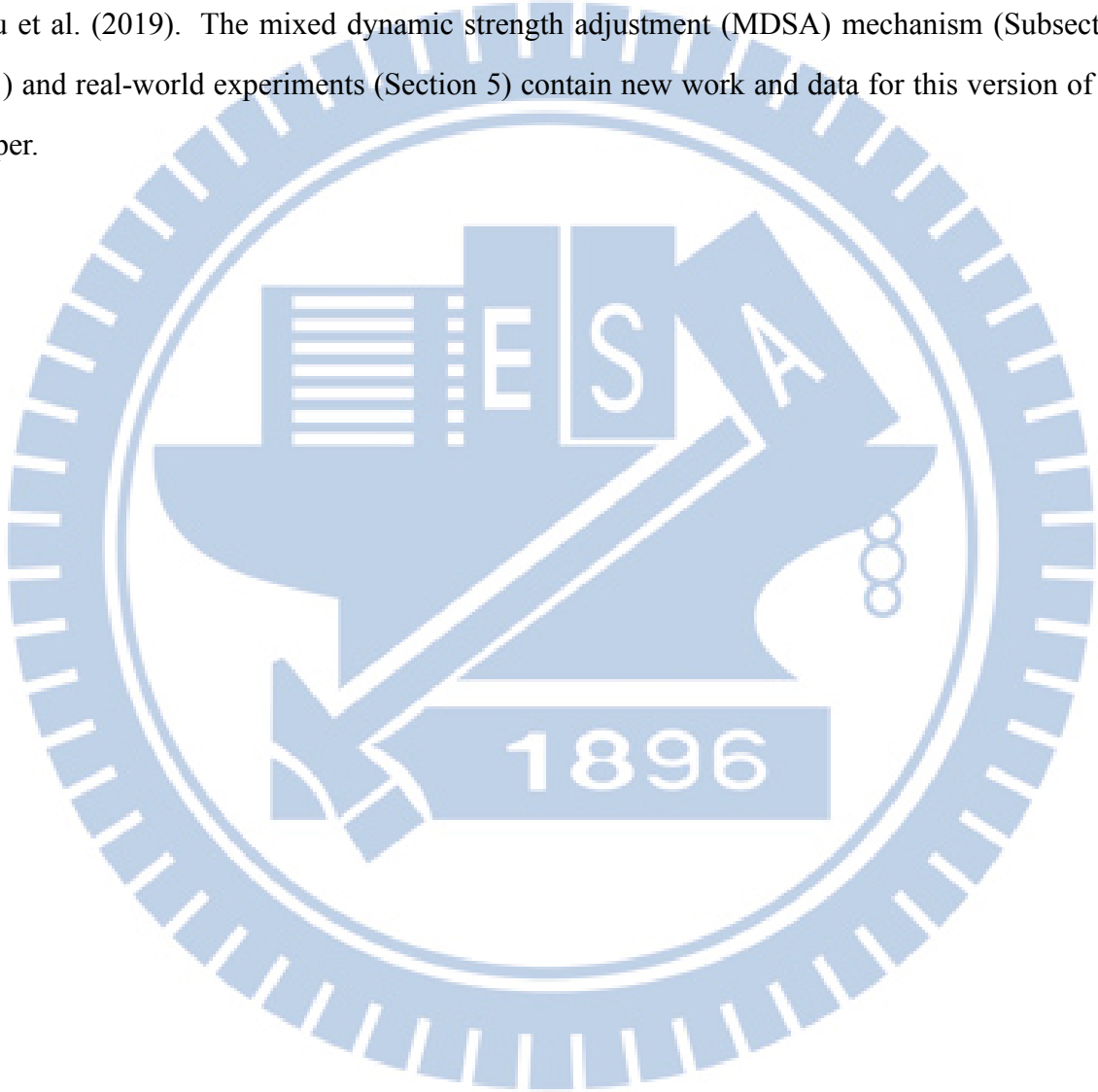
The second straightforward approach is to offer different programs for different strength levels. For example, we could train separate DNNs for each difficulty. A good example of this type of strength adjustment is Paulsen and Frnkranz’s work¹ on training chess evaluation functions for different strengths Paulsen and Frnkranz (2010). However, this approach usually requires large amounts of time and effort to tune and test the programs. This is especially difficult for games like Go, where strength levels span a wide range from 30 kyu to professional 9 dan Arno and Morten (2015), corresponding to about a 3000 Elo rating Elo (1978) difference.

This paper reviews a strength adjustment (SA) approach based on the softmax policy and proposes our modification in Section 2. We then apply the method to two programs: 1) the open source Go program ELF OpenGo (abbr. ELF for the remainder of this paper); 2) the Go program CGI Shih et al. (2018). From these two implementations, we demonstrate that the modified strength adjustment method can be easily used to adjust the strength of a program, covering

¹Paulsen and Frnkranz’s method does not use DNNs, but is similar to our example here in that they create multiple programs, each offering one static strength.

a range of over 800 Elo ratings. In Section 3, this paper presents a hypothesis and performs theoretical analyses to justify the empirical strengths shown in Section 2. Having demonstrated that the program strength can be adjusted with relative ease, we introduce methods to adjust the strength dynamically in Section 4. In Section 5, we test our method on real-world scenarios, playing against human players. Finally, we summarize our contributions in Section 7.

This paper is an extension of our work, which was published in the AAAI 2019 conference Wu et al. (2019). The mixed dynamic strength adjustment (MDSA) mechanism (Subsection 4.3) and real-world experiments (Section 5) contain new work and data for this version of the paper.



II . Static Strength Adjustment (SSA)

In this section, we first review past work on strength adjustment, then present our modifications to the method. We apply the modified approach to the Go program ELF and provide empirical data.

2.1 Past Work

For strength adjustment, Sephton et al. Sephton et al. (2015) investigated some selection mechanisms for MCTS-based game-playing programs. One of these mechanisms uses a simple softmax policy as follows. Given strength index z , choose moves i with probability $N_i^z / \sum_j N_j^z$, where N_i is the number of simulations on move i in MCTS. For simplicity of discussion in the rest of this paper, let $N_i \geq N_j$ if $i < j$, i.e. N_1 is the maximum.

Conceptually, z is the inverse of the softmax temperature. When z is higher, the policy tends to choose moves with higher simulation counts, which tends to be a higher-quality move as is the case with MCTS. When z approaches infinity, the move with the highest simulation count is guaranteed to be chosen, and thus the policy exhibits the same behavior as the original MCTS. When $z = 0$, all moves are chosen with equal probability, i.e., moves are chosen randomly. When z approaches negative infinity, the move with the lowest simulation count is chosen, i.e. the policy tends to choose the lowest quality moves.

Thus, z can serve as an index of strength. Sephton et al. Sephton et al. (2015) showed through experiments that z is correlated to the empirical strength. In their experiments, the strength indices only covered six trials $z \in \{1, 2, 3, 4, 5, 6\}$ for the game Lords of War, where the differences of win rates for these values of z range from 5% to 24%, equivalent to a range of about 100 Elo ratings.

As stated above, when z is low, the policy tends to choose low quality moves. However, in MCTS, many moves are not visited during simulation, or in some cases, visited very few times only because of the exploration bias Kocsis and Szepesvári (2006). For this reason, it is not a good idea to allow the policy to choose the lowest-quality moves, which would result in a much weaker program or unpredictable behavior.

In order to avoid choosing the very lowest-quality moves, Sephton et al. suggested choosing the first n best moves as candidates, where n is a given fixed value. However, it is still possible to choose a very low-quality move, e.g., in the case that only one move is viable while the others are extremely bad, the policy is still likely to choose bad moves.

2.2 Our Approach

In our approach, we follow the softmax policy to choose moves via the strength index z . With a simple softmax policy, there is the possibility of selecting poor moves. To perform move screening and in turn ensure move quality, we use a threshold ratio R_{th} to avoid moves with lower simulation counts. Namely, given a threshold ratio R_{th} , we only consider the moves i with $N_i \geq N_1 \times R_{th}$ as candidates. Assuming that the move quality is correlated to the simulation count (we discuss this in this in greater detail in Section 3), this approach ensures that the qualities of the chosen moves are higher than the screened moves, which do not reach the threshold. At the very least, the modified policy is less likely to choose extremely bad moves, as Sephton et al.’s method Sephton et al. (2015) is prone to do.

For a high threshold ratio, more low-quality moves are filtered. When $R_{th} = 1$, the move with the highest simulation count is always chosen, behaving the same way as the original MCTS. In contrast, for a low threshold ratio, many low-quality moves are not filtered. Thus, it is important to set a reasonable threshold ratio, where the goal is to filter most low-quality moves, while simultaneously allowing reasonable moves to be considered.

In contrast to Sephton et al.’s method Sephton et al. (2015), our empirical results in the next section show that strengths can be adjusted across a wide range over 800 Elo ratings with the threshold ratio set to 0.1 and $z = [-2, 2]$. Thus, our approach is very suitable for games that are considered to have very high depth Cauwet et al. (2015).

Table 1: The win rates (against ELF with $z = 0$) and Elo ratings (relative to the original ELF) with respect to z when $R_{th} = 0.1$.

z	Win rate ($\pm$ errors)	Elo rating ($\pm$ errors)
∞	97.6% ($\pm 1.9\%$)	0 (-106, +289)
2	94.4% ($\pm 2.9\%$)	-153 (-78, +133)
1.5	92.4% ($\pm 3.4\%$)	-210 (-70, +107)
1	91.2% ($\pm 3.6\%$)	-237 (-66, +98)
0.5	71.6% ($\pm 5.7\%$)	-483 (-46, +52)
0	50.0%	-644
-0.5	35.6% ($\pm 6.1\%$)	-747 (-48, +44)
-1	21.6% ($\pm 5.2\%$)	-868 (-59, +49)
-1.5	13.2% ($\pm 4.3\%$)	-971 (-76, +58)
-2	12.4% ($\pm 4.2\%$)	-983 (-79, +59)
$-\infty$	7.2% ($\pm 3.3\%$)	-1088 (-111, +71)

2.3 Empirical Evaluation

We apply the above approach to the Go program ELF Yuandong Tian et al. (2019) with the 20-block model and present the experiment results. All the experiments are performed on machines equipped with one GTX 1080Ti GPU, one Intel Xeon E5-2683 v3 (14 cores in total), 2.6 GHz, 128 GB memory, and with Linux. All games are played with one second per move, using one GPU and six CPU cores. For each benchmark, 250 games are played against a baseline consisting of ELF with $R_{th} = 0.1$ and $z = 0$. Note that the original ELF is equivalent to the setting with $z = \infty$, which we do not use as the baseline since it is much too strong for some trials, such as when $z = -\infty$.

Table 1 shows the win rates and the relative Elo ratings of the ELF versions with $R_{th} = 0.1$ and with different values of z against the baseline. The shown Elo ratings are relative to the original ELF ($z = 0$) for simplicity of analysis. Since ELF follows the process of training AlphaGo Zero with 20 residual blocks, its Elo rating of the ELF version is expected to be between 4000 and 5000 based on AlphaGo Zero Silver et al. (2017b).

Figure 1 shows the correlation between z and the Elo ratings. Interestingly, both are highly correlated with a low linear regression error of 47.95 Elo, in terms of the Elo rating, when z is between -2 to 2. In addition, the range of strength is very wide, covering 1088 Elo rating for all z and 830 for the interval of z in $[-2, 2]$.

Furthermore, Figure 2 depicts the correlation between z and the Elo rating for different

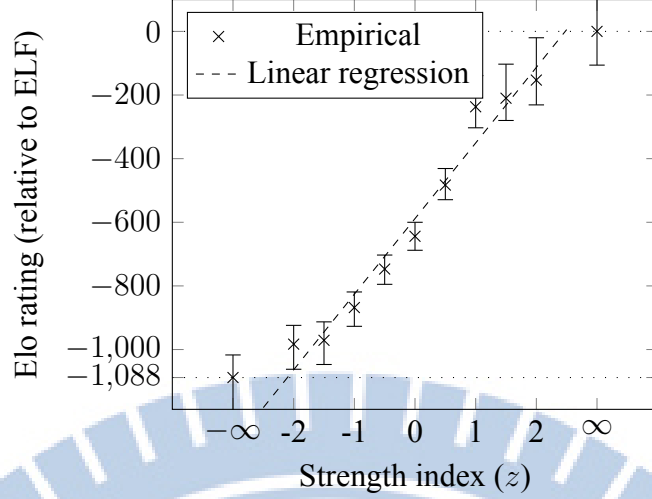


Figure 1: The correlation between z and Elo rating with $R_{th} = 0.1$.

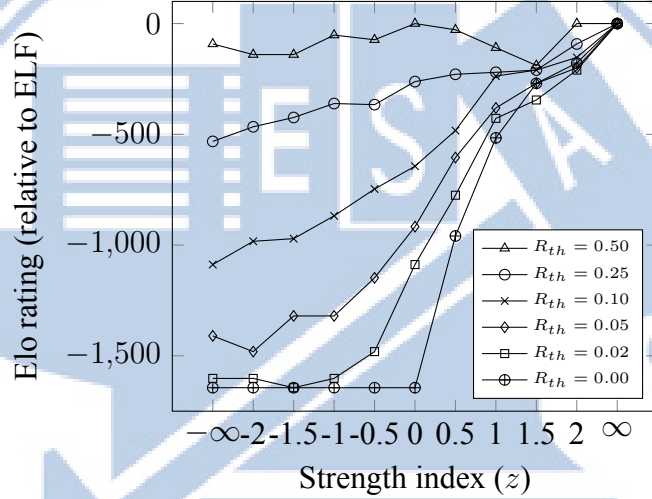


Figure 2: Elo rating (relative to ELF) in different threshold ratios and strength indices.

threshold ratios, 0, 0.02, 0.05, 0.1, 0.25, and 0.5. All games are also played against the same baseline as above. From Figure 2, the correlation between Elo ratings and z is also highly correlated to R_{th} in most cases. A higher value of z tends to correspond to higher Elo ratings.

We observe that high values of R_{th} do not exhibit the intended strength adjustment effects. For example, when $R_{th} = 0.5$, the Elo rating has no significant changes across different values of z . An intuitive explanation for this is that with a high threshold ratio, most candidate moves are filtered, so the value of z does not matter as much. Figure 3 shows that the average number of candidates is only 1.4 for $R_{th} = 0.5$, and 1.9 for $R_{th} = 0.25$. Another effect is that the adjusted strength range is narrower, e.g., smaller than 500 Elo rating for $R_{th} = 0.25$.

On the other hand, for low threshold ratios, the Elo rating drops quickly, and the strength

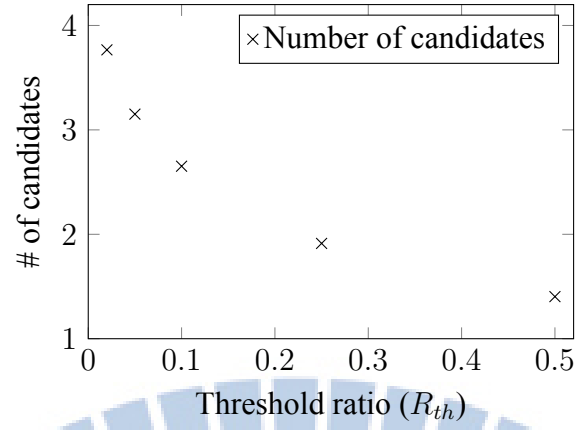


Figure 3: The number of candidate moves with respect to R_{th} .

for different values of z show no difference, e.g. when $R_{th} = 0$ and $z \leq 0$, and when $R_{th} = 0.02$ and $z \leq -1$.

Thus, judging from both Figure 2 and Figure 3, the threshold ratios of 0.05 and 0.1 appear to be suitable for our needs. For simplicity of analysis, 0.1 will be used as the threshold ratio, unless otherwise stated.

III . Theoretical Analysis

The above empirical results show that the strengths are highly correlated to z . In fact, between $-2 \leq z \leq 2$ and a threshold ratio of 0.1, z and the strength show a near linear relationship with regression error 47.95 Elo. However, the strength or Elo rating should be fixed when z approaches ∞ or $-\infty$. Thus, intuitively, the curve of the Elo rating strength according to the z value should be shaped similar to a logistic function. Applying logistic regression Hosmer Jr et al. (2013), the curve is close to a logistic function with error 26.00 Elo ($\beta_0 = -0.25, \beta_1 = 1.16$).

This section investigates this conjecture of logistic regression from a theoretical perspective. First, we review the generalized Bradley-Terry model. Second, we present a hypothesis on move strength. Then, from theoretical analysis, we show that the derived strengths are close to the empirical strengths. We calculate the regression error between the derived and the empirical strengths to justify the hypothesis.

3.1 Generalized Bradley-Terry Model

The Bradley-Terry model has been the foundation of various ranking systems, including the Elo rating system. The model is used to estimate the strengths of players and predict the win rates among these players. Namely, each player i is associated with a positive value γ_i representing the strength of i , and the probability that i wins over j is $\gamma_i/(\gamma_i + \gamma_j)$. Obviously, the higher γ_i is, the higher the winning rate (implying a stronger player). The Elo rating of individual i is $Elo_i = 400 \log_{10} \gamma_i$ Coulom (2008). For simplicity of discussion in this paper, we also define the rating $E_i = \ln \gamma_i$, whose corresponding Elo rating is $Elo_i = 400(\log_{10} e)E_i$.

The Bradley-Terry model has also been generalized to handle competitions involving more

than two players Coulom (2007) Hunter et al. (2004). Namely, the probability that i wins among n players, $1, \dots, n$, is formulated as $\gamma_i / \sum_i \gamma_i$. Another generalization is to allow competitions among teams, instead of players. The strength and its corresponding rating of a team of n players is estimated as

$$\gamma^{all} = \prod_i^n \gamma_i, \text{ and } E^{all} = \sum_j^n E_j \quad (3.1)$$

In this paper, we also define the average strength and rating of a team of n players to be

$$\gamma^{avg} = (\prod_i^n \gamma_i)^{\frac{1}{n}}, \text{ and } E^{avg} = (\frac{1}{n}) \sum_j^n E_j \quad (3.2)$$

This is useful when n is not fixed. In addition, consider a team that can choose one and only one player to participate and choose player i with probability π_i , where $\sum_j^n \pi_j = 1$. Thus, the strength and rating of the team are

$$\gamma = \prod_i^n \gamma_i^{\pi_i}, \text{ and } E = \sum_j^n \pi_j E_j \quad (3.3)$$

respectively, for the reason as illustrated below. For example, let $\pi_i = N_i / \sum_j^n N_j$. We can consider the team composed of $\sum_j^n N_j$ players, among which the number of players i is N_i . Thus, the average strength and rating of the team are the same as γ and E in Equation (3.3), respectively.

3.2 Hypothesis

As mentioned above, moves with higher simulation counts N_i in MCTS normally tend to have higher quality. Following this notion, we present a hypothesis for further theoretical analysis. Assume that given a position the strength of move i is proportional to N_i^H . Here, H denotes a conjectured strength index for moves to be selected in MCTS in the previous sections. Namely, let $\gamma_i = c \times N_i^H$. Here, c is a constant coefficient with respect to the same game position, i.e. different positions may have different relative strengths, and therefore will have a different value of c .

If we view the moves as separate players that follow the generalized Bradley-Terry model, the rating of move i is $E_i = \ln \gamma_i = \ln c + H \ln N_i$. For simplicity of analysis, we use E_i in the following analysis without loss of generality. If the Elo rating is preferred, Elo_i can be obtained by a simple conversion, as described above.

Let $\gamma(z)$ and $E(z)$ denote the overall strength and rating following the above method for strength adjustment, which chooses among all moves i using the softmax policy $\pi_i(z) = N_i^z / \sum_i (N_i^z)$. From the above Bradley-Terry model for team strength, we can derive that

$$\gamma(z) = \prod_i \gamma_i^{\pi_i(z)}, \text{ and} \quad (3.4)$$

$$\begin{aligned} E(z) = \ln \gamma_i &= \sum_i \pi_i(z) E_i \\ &= \sum_i \pi_i(z) (\ln c + H \ln N_i) \\ &= \ln c + H \sum_i \pi_i(z) \ln N_i \end{aligned} \quad (3.5)$$

In the above formula, the first item in Equation (3.5) is fixed for this position, and therefore it can be removed to obtain relative ratings, say, relative to the rating where $z = \infty$ (which always chooses the move with the maximum simulation count N_1) as follows.

$$\begin{aligned} E_{rel}(z) &= E(z) - E(\infty) \\ &= H \sum_i \pi_i(z) (\ln N_i - \ln N_1) \\ &= H \sum_i \pi_i(z) (\ln R_i), \end{aligned} \quad (3.6)$$

where R_i is the ratio N_i/N_1 . Since all moves with the ratio less than R_{th} are filtered out, $R_i \geq R_{th}$. In addition, since N_1 is the maximum among N_i , $R_i \leq 1$ and $\ln R_i$ are therefore all non-

positive. Thus, we obtain

$$E_{rel}(z) \geq H \times (\ln R_{th}). \quad (3.7)$$

An important implication in Equation 3.7 is that the relative ratings of the chosen moves are at worst $H \times (\ln R_{th})$. Assume $R_{th} = 0.1$. The relative ratings of all chosen moves are at worst $H \times \ln 0.1 \cong -2.3H$, no worse than move 1 by $2.3H$. Since H is a constant under this hypothesis, this implies that the strength of any chosen move is at worst a fixed value. This ensures the quality of all chosen moves.

Now, let us consider following the above SSA method to play a game g , containing a sequence of m_g moves or m_g positions to move. Let $\gamma^{(j)}(z)$ and $E^{(j)}(z)$ denote the strength and rating of the move made at the j_{th} position (i.e. on the j_{th} turn), which can be formulated as follows.

$$\gamma^{(j)}(z) = \prod_i \gamma_i^{\pi_i(z)}, \text{ and} \quad (3.8)$$

$$\begin{aligned} E^{(j)}(z) &= \sum_i \pi_i^{(j)}(z) E_i^{(j)} \\ &= \ln c^{(j)} + H \sum_i \pi_i^{(j)}(z) \ln N_i^{(j)} \end{aligned} \quad (3.9)$$

where $\gamma_i^{(j)}$, $E_i^{(j)}$, $\pi_i^{(j)}$ and $N_i^{(j)}$ are respectively the strength, rating, policy and simulation count of moves i at the j_{th} position in the game, and $c^{(j)}$ is the coefficient with respect to the position.

Furthermore, let $\gamma^g(z)$ and $E^g(z)$ denote the averaged strength and rating as follows.

$$\gamma^g(z) = \left(\prod_j \gamma^{(j)}(z) \right)^{1/m_g}, \text{ and} \quad (3.10)$$

$$\begin{aligned}
E^g(z) &= \left(\frac{1}{m_g}\right) \sum_j E^{(j)}(z) \\
&= \left(\frac{1}{m_g}\right) \sum_j (\ln c^{(j)} + H \sum_i \pi_i^{(j)}(z) \ln N_i^{(j)}) \\
&= \left(\frac{1}{m_g}\right) \sum_j \ln c^{(j)} + \left(\frac{H}{m_g}\right) \sum_j \sum_i \pi_i^{(j)}(z) \ln N_i^{(j)}. \tag{3.11}
\end{aligned}$$

Note that we evaluate the averaged strength and rating as Equation (3.1), instead of the aggregated values in Equation (3.2), simply because the number of moves in a game is not fixed.

In the above formula, the first item is fixed, and therefore can be omitted when calculating ratings relative to the one with $z = \infty$, similarly, as follows.

$$\begin{aligned}
E_{rel}^g(z) &= E^g(z) - E^g(\infty) \\
&= \left(\frac{H}{m_g}\right) \sum_j \sum_i \pi_i^{(j)}(z) (\ln R_i^{(j)}), \tag{3.12}
\end{aligned}$$

where $R_i^{(j)} = N_i^{(j)} / N_1^{(j)}$. Moreover, let the relative rating be normalized to be independent of the value H as follows.

$$\begin{aligned}
E_{norm}^g(z) &= E_{rel}^g(z) / H \\
&= \left(\frac{1}{m_g}\right) \sum_j \sum_i \pi_i^{(j)}(z) (\ln R_i^{(j)}) \\
&= \mathbb{E}_g \left[\sum_i \pi_i^{(j)}(z) (\ln R_i^{(j)}) \right]. \tag{3.13}
\end{aligned}$$

For stochastic analysis, we extend by collecting some sets of games, each of which is collected from the games under a designated threshold ratio in the above empirical experiments. We exclude extreme cases to minimize the effect of noise for our analysis. For example, the cases of $z = \infty$ and $z = -\infty$ are not included.

For simplicity of analysis, let us illustrate the case for $D_{0.1}$, denoting the set of games with

threshold ratio 0.1, which contains about 2000 games. The expected relative rating under the set $D_{0.1}$ is

$$\begin{aligned}
E_{norm}^{D_{0.1}}(z) &= \mathbb{E}_{g \sim D_{0.1}}[E_{norm}^g(z)] \\
&= \mathbb{E}_{g \sim D_{0.1}}[\mathbb{E}_g[\sum_i \pi_i^{(j)}(z)(\ln R_i^{(j)})]] \\
&= \mathbb{E}_{g \sim D_{0.1}}[\sum_i \pi_i^{(j)}(z)(\ln R_i^{(j)})].
\end{aligned} \tag{3.14}$$

Figure 4 depicts the solid curve of $E_{norm}^{D_{0.1}}(z)$ calculated from the set $D_{0.1}$ according to Equation (3.14). The left y axis indicates the value of $E_{norm}^{D_{0.1}}(z)$. The curve resembles a logistic function. Now, let $E^{D_{0.1}}(z)$ denote the expected rating, and $H^{D_{0.1}}$ be the value H , under the set of games $D_{0.1}$. Thus, we have

$$E^{D_{0.1}}(z) = E^{D_{0.1}}(\infty) + H^{D_{0.1}} E_{norm}^{D_{0.1}}(z). \tag{3.15}$$

Then, we can derive that

$$E^{D_{0.1}}(-\infty) = E^{D_{0.1}}(\infty) + H^{D_{0.1}} E_{norm}^{D_{0.1}}(-\infty), \text{ and} \tag{3.16}$$

$$H^{D_{0.1}} = \frac{E^{D_{0.1}}(-\infty) - E^{D_{0.1}}(\infty)}{E_{norm}^{D_{0.1}}(-\infty)} \tag{3.17}$$

Since the values $E^{D_{0.1}}(\infty)$ and $E^{D_{0.1}}(-\infty)$ are supposed to approximate the strength in the empirical experiments, they can be replaced by the empirical strengths at $z = \infty$ and $z = -\infty$, whose relative Elo ratings are 0 and -1088 as shown in Table 1. Thus, the value $H^{D_{0.1}}$ is derived to be 6.243 according to the above formula. The right y axis in Figure 4 follows the y axis in Figure 1. The regression error to the empirical strengths for z between -2 and 2 is about 40.45 Elo, and the regression error to a logistic regression curve is about 10.51 Elo ($\beta_0 = -0.05, \beta_1 = 1.17$). These low errors justify the hypothesis.

In our experiments, we also derived the value H for other sets of games, as shown in Table 2. From the table, $H^{D_{0.25}}$ is almost the same as $H^{D_{0.1}}$, while $H^{D_{0.5}}$, $H^{D_{0.05}}$ and $H^{D_{0.02}}$

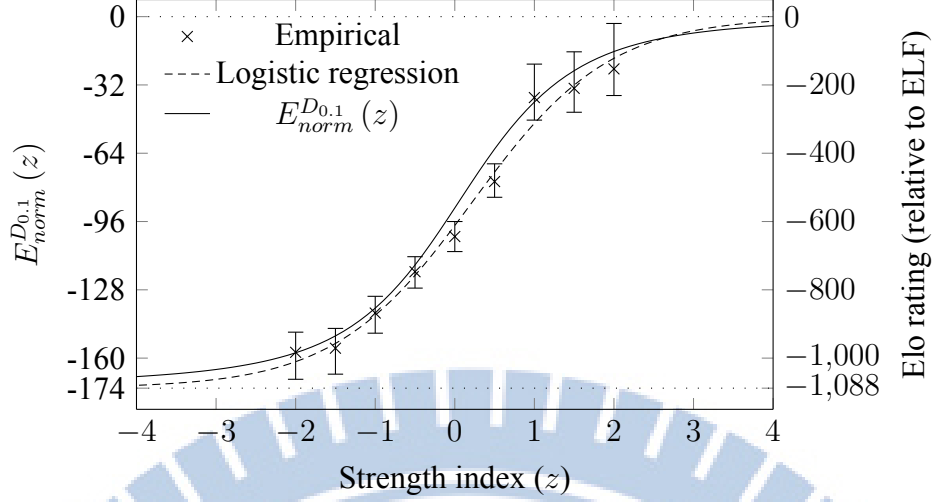


Figure 4: The curve of $E_{norm}^{D_{0.1}}(z)$ and the empirical data.

Table 2: The conjectured strength indices estimated in different data sets.

Data set	$D_{0.02}$	$D_{0.05}$	$D_{0.1}$	$D_{0.25}$	$D_{0.5}$
H	4.385	5.369	6.243	6.244	3.292

are lower. For $H^{D_{0.05}}$ and $H^{D_{0.02}}$, our conjecture is that the noise incurred from having a low threshold ratio are high as the following illustration. In the case of $R_{th} = 0.02$, since the average number of simulation counts for the best move is about 259.4 with the one second time limit, it is highly likely to include the moves with very low simulation counts (the threshold is about $259.4 \times 0.02 \cong 5.2$). Since many of these simulations may be generated simply because of the exploration bias, these simulations may introduce noise and therefore affect the verification of our hypothesis.

As for $H^{D_{0.5}}$, we observe that the average number of candidates is 1.4 from Figure 3. Since the number is relatively low, in many cases, the policy chooses only from a single candidate move. Therefore, the distribution is insufficient to justify our hypothesis. As an example, the most extreme case is where the threshold ratio is 1, and only the moves with the highest simulation counts are chosen, as in the original MCTS. The value of H in this case does not affect the policy at all, since there is only one choice.

IV . Dynamic Strength Adjustment (DSA)

As stated in the previous sections, this paper presents a flexible strength adjustment method simply by altering the value z with an appropriate R_{th} , say 0.1. Moreover, the strength ratings are approximately linear with respect to z in the interval $[-2, 2]$. This allows us to fit the programs strength to its opponents' dynamically, provided the opponents' strengths are within Elo rating differences of $[-983, -153]$ (compared to the original ELF), corresponding to the range of z in $[-2, 2]$ (shown in Figure 2, $R_{th} = 0.1$). This section introduces two types of dynamic strength adjustment, inter-game and intra-game strength adjustment. For the former, strengths are adjusted between multiple games based on previous game results, while for the latter strengths are adjusted within each game. We present two methods of dynamic strength adjustment (DSA) to showcase how we can predict opponent strengths and adjust accordingly with relative ease. There are many ways to design DSA mechanisms; the presented methods are by no means a comprehensive review of all available methods.

4.1 Inter-game Strength Adjustment

Inter-game strength adjustment is relatively easy. Namely, the strength index z of a game is adjusted based on the previous game results and the index remains unchanged within the game.

In this section, a simple adjustment method is presented and demonstrated to predict the opponent' s strength. The prediction can then be used to set z accordingly. The strength index z is decreased for every win and increased for every loss, both by a small amount Δz . The initial value of z is set to 0. The value Δz is initialized to Δz_{init} and decreased by a discount factor r

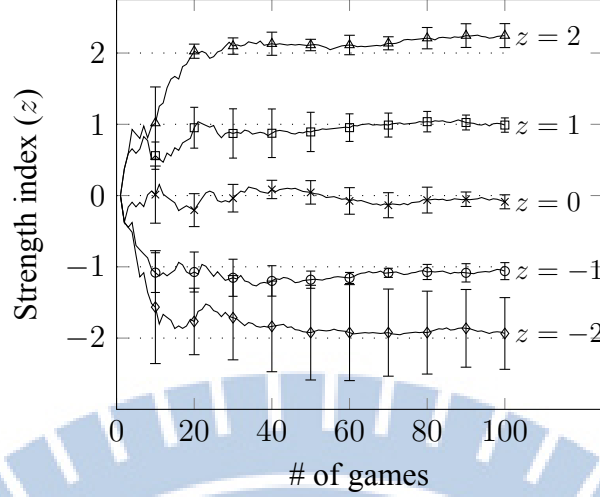


Figure 5: Strength index estimation for inter-game SA.

Table 3: Win rate (WR) and average dynamic strength index (Avg. z) against different opponents using inter-game SA.

Opponent	$z = 2$	$z = 1$	$z = 0$	$z = -1$	$z = -2$
w/o DSA WR	5.6% ($\pm 2.9\%$)	8.8% ($\pm 3.6\%$)	50.0%	78.4% ($\pm 5.2\%$)	87.6% ($\pm 4.2\%$)
Inter-game WR	43.4% ($\pm 4.4\%$)	46.6% ($\pm 4.5\%$)	52.0% ($\pm 4.5\%$)	50.0% ($\pm 4.5\%$)	54.8% ($\pm 4.5\%$)
Avg. z	1.93	0.88	-0.04	-1.06	-1.73

for each game, capped by a lower bound Δz_{low} .

In our experiments for the method, Δz_{init} is 0.375, approximately equivalent to 100 in Elo rating based on the linear regression in Figure 1, then decreased by a factor of $r = 0.95$ for each game, with $\Delta z_{low} = 0.03$, approximately equivalent to 8 in Elo rating. In the experiment, 100 games are played against each of the five opponents whose strength indices are $z = 2, 1, 0, -1, -2$ for a total of 500 games. The experiment is repeated five times and the following experiment results are based on the average values of the five times.

In Figure 5 each of the five lines indicates the predicted z for each opponent. The result shows that our method can approximately predict opponents' strengths and clearly distinguish five opponents after 10 games. Table 3 also shows that the averaged win rate for each opponent is within 43% $\sim$ 54% and the averaged predicted z is very close to the opponent's.

4.2 Intra-game Strength Adjustment

Intra-game strength adjustment is relatively challenging, given that the algorithm only has one game to predict the opponent's approximate level of play. Players often play inconsistently, mixing objectively good and bad moves within the same game. On the one hand, adjusting by large amounts leads to high variance of program strength. On the other hand, if strengths are adjusted by a small amount, the effects may not be sufficiently obvious.

Our method is as follows. In principle, we still attempt to maintain all moves so that the overall win rate is around 50%. For each move, we first estimate the current win rate W , by using the MCTS win rate of the move with the most simulation counts. The index z is decreased when $W > 50\%$ and increased when $W < 50\%$, both by a value of Δz . The program chooses moves based on the softmax policy proposed earlier.

For stability, Δz is set to be relatively small when W is within a range $(50\% - \epsilon, 50\% + \epsilon)$, where ϵ is a user defined value, say 10%. Namely,

$$\Delta z = \begin{cases} \Delta Z, & \text{if } |W - 50\%| \geq \epsilon \\ \Delta Z \times \frac{|W - 50\%|}{\epsilon}, & \text{if } |W - 50\%| < \epsilon \end{cases} \quad (4.1)$$

In addition, ΔZ decreases linearly from an initial value, say 0.1, to 0 after a number of moves, say 150 moves. The purpose is to cool down the magnitude of changes as games progress, since the program should have an idea of its opponent's strength by later stages in the game. This cool down mechanism is important because without it, the program will make unreasonable concessions when it is in the lead, or ramp up in strength indefinitely when it is behind.

In the experiment, the above method is used to play against five opponents with strength indices $z = 2, 1, 0, -1$, and -2 . For each opponent, we consider two cases of initial ΔZ , $\Delta Z_{init} = 0.2$ and 0.1 , where for each case, 100 games are played.

Table 4 presents the experiment results. The results show that the average z values over 100 games are close to the opponents' strength indices, especially when $\Delta Z = 0.2$. Note that the predicted z for each game is the value z at the end of the game. The standard deviation is

Table 4: Win rate (WR), average z (Avg. z) and standard deviation of z (Std. z) against different opponents using intra-game SA.

Opponent		$z = 2$	$z = 1$	$z = 0$	$z = -1$	$z = -2$
w/o DSA		5.6%	8.8%	50.0%	78.4%	87.6%
$\Delta z_{init} = 0.2$	WR	29.0%	38.0%	43.0%	64.0%	71.0%
	Avg. z	1.85	0.92	-0.10	-1.39	-1.57
	Std. z	1.82	1.81	1.41	1.73	1.56
$\Delta z_{init} = 0.1$	WR	15.0%	32.0%	49.0%	73.0%	72.0%
	Avg. z	1.02	0.75	-0.21	-0.66	-0.96
	Std. z	0.94	0.90	0.81	0.84	0.85

high as expected.

The results in Table 4 also show that while all the win rates except for $z = 0$ are not around 50%, when compared to the baseline win rates without DSA (as shown in the second row), the overall win rates are closer to 50%. This shows that intra-game DSA can predict opponents' strengths and even out the games. The reason why the win rates are not balanced around 50% despite the predicted z to be more or less accurate, is that the early moves in a game influence the outcome significantly, but the program has yet to observe its opponents' strengths sufficiently at that point.

4.3 Mixed Dynamic Strength Adjustment (MDSA)

We can expect many different variants for DSA by, say, modifying hyper-parameters for the above methods, or by using hybrids of inter-game and intra-game DSA. For example, we can use intra-game DSA for the first few games in a series to make rapid adjustments to the right strength, then follow up by applying inter-games DSA to arrive at the strength more accurately for the remaining games in the series. A hybrid case is demonstrated in the following experiment: we first apply intra-game DSA to the first two games with $\Delta Z_{init} = 0.2$, then apply inter-game DSA to the remaining games with the same settings as those in Figure 6. Figure 6 presents the experimental result which shows a fast convergence to opponents' strengths, especially when $z \in \{1, -1, -2\}$. In summary, the above MDSA method uses intra-game SA in the beginning to get a rough strength estimate, then follows up by using inter-game SA to fine tune the player's strength estimation.

Table 5: Win Rate (WR) and average dynamic strength index (Avg. z) against different opponents using MDSA.

Opp.	$z = 2$	$z = 1$	$z = 0$	$z = -1$	$z = -2$
WR	45.0%	46.0%	46.7%	48.0%	54.3%
Avg. z	1.69	0.76	-0.10	-1.21	-1.83

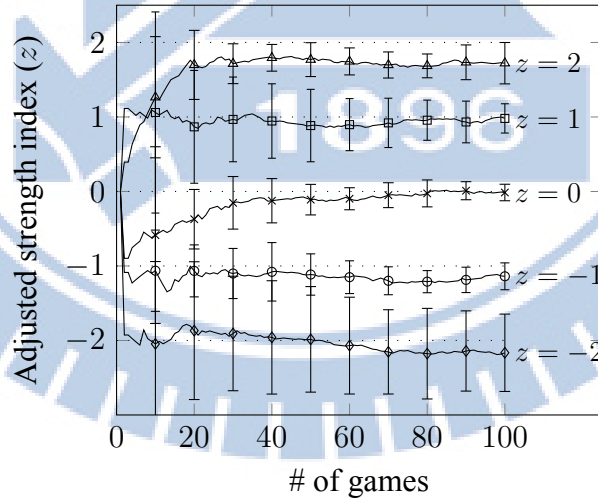


Figure 6: Strength index estimation for MDSA.

V . Real-World Experiments

In this section, we use SSA and DSA to test them against human players of various playing abilities. We applied strength adjustment to three versions of our Go program CGI Wu et al. (2018), 3.0, 2.0 and 1.0, which can cover a strength ranging over 2800 Elo ratings, listed as follows from strongest to weakest: professionals, dan rank amateurs, and kyu rank amateurs. For professionals, we use CGI 3.0, consisting of both the policy network and the value network (as is the case with AlphaGo Zero Silver et al. (2017b)), with settings that follow AlphaGo’s Silver et al. (2016), where the simulation time was set to 20 seconds. For the amateur level 1 dan to 7 dan, CGI 2.0 uses only the policy network (without the value network), also with settings following AlphaGo’s Silver et al. (2016), with 4 seconds of simulation time. For amateur 18 kyu to 1 kyu (weaker than the dan ranks), CGI 1.0 uses the traditional minorization-maximization method Coulom (2007) (without neural networks) with 1 second simulations.

5.1 Static Strength Adjustment

First, to verify that our strength adjustment is able to provide a steady range of program strengths, we perform the following experiment. Let us assume there is a fair and accurate rating system for a list of human Go players, serving as a baseline system (e.g., the Go rating site Coulom (2019)), where each player has a rating. As the human players play against a range of strength-adjusted programs, we can obtain a series of game records which form the basis of a separate rating system. If the ratings obtained by this separate rating system is highly correlated with the baseline ratings, we can conclude that the programs have stationary strengths, and that the strengths must range from the weakest human player to the strongest.

For this experiment, six different strength indices $z \in \{1.1, 1.4, 1.7, 2.0, 2.5, 3.0\}$ for CGI

3.0 were tested against 15 professional Go players from HaiFong Go Association HaiFong Go Association (2018), the biggest Go association in Taiwan. The names of the players are listed in Table 8. A total of 167 games were played for game record collection, where each player contributed to at least 8 games. Although not enforced, the professional players were instructed to adjust his/her opponent according to their experience with the programs; i.e. pick a stronger one if he/she wins, or vice versa. For the baseline, we used ratings from the most widely used system for professional Go players, the Go Ratings website Coulom (2019). The website uses the whole-history rating (WHR) rating system Coulom (2008) to estimate player strengths. The second set of ratings derived from human-program game records also follow the WHR algorithm.

The results are shown in Figure 7. The correlation between the human-program ratings and the ratings obtained from Go Ratings is high ($R^2 = 0.6661$, note that R^2 is R-squared values). There are a wide variety of reasons that may affect human player performance. For example, for this first experiment, we did not control for the environment in which the professional players played against our program. Their performance may vary depending on if they played against our program in their own home or in at the Go association, which is more professional. Another difference is the timing system used. The players were free to choose their preferred time system, ranging from 1 to 10 minutes of main time; byo-yomi (overtime) is uniformly set to three 30 seconds intervals. Ideally, we should eliminate these factors, but to minimize the disruption to the professional players' training schedule, we had to use the collected records as best as we could for analysis. This may explain outliers such as P14 and P17 in Figure 7.

During these 167 games, we also collected survey responses following the work in Silva et al. (2017) on dynamic difficulty adjustment for video games. The survey asked participants about the perceived strength of the opponent, entertainment level, and any other general comments.

We first annotate general comments before analyzing numerical survey results. In regards to game balancing and perceived strength:

- P4: White (CGI)'s moves at 48 and 50 are interesting, from 58 to 66, white played wonderfully, winning the game at 94.

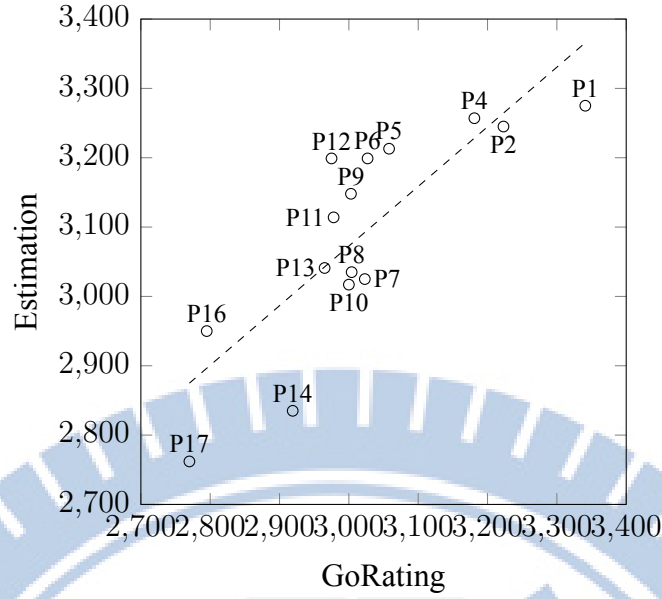


Figure 7: SSA HaiFong professional player rating estimation using WHR (see Table 8 for player name references).

- P6: This version does not seem to be too strong ($z = 1.1$). This one is stronger ($z = 1.4$).
- P8: It feels like this game ($z = 2.5$) has a sizeable gap in strength (stronger) compared to the previous game ($z = 2$).
- P9: The moves at 20, 24 (CGI) are plays that have not been seen in any human game records.

In terms of quantifiable survey results, we construct a normalized contingency table, as shown in Table 6. The wording on the survey instructs the player to give a perceived strength rating from 1 to 9 (weakest to strongest), relative to her own playing strength. First, we can see that in terms of entertainment value, professional players tend to enjoy opponents that are slightly stronger than themselves (for entertainment scores of 7 to 9). For opponents that are much weaker than the players, the entertainment value is rated worst with no exceptions. In terms of strength, it is interesting to observe that for the strongest relative opponents (perceived strength of 9), there are two extremes. Some find highly challenging opponents to be very entertaining, while others do not enjoy playing against opponents that are far stronger than themselves.

Next, we collect data for amateur players by collaborating with the two largest online Go platforms (in terms of players): Tygem Tongyang Online (2019) and Fox Weiqi Tencent (2019) (abbr. FoxWQ for the remainder of this paper). Both websites provide AI developers with

Table 6: Normalized Contingency Table (row).

		Perceived strength								
		1	2	3	4	5	6	7	8	9
Entertainment	1	0.1	0.1	0	0	0.2	0	0	0.2	0.4
	2	0	0	0.11	0.11	0.11	0.11	0.11	0.22	0.22
	3	0	0	0.12	0.12	0.18	0.24	0.24	0.12	0
	4	0	0	0.05	0.09	0.41	0.23	0.18	0.05	0
	5	0	0	0.05	0.1	0.43	0.33	0.08	0	0.03
	6	0	0	0	0.18	0.32	0.24	0.18	0.03	0.06
	7	0	0	0.02	0.12	0.24	0.26	0.3	0.04	0.02
	8	0	0	0	0	0.13	0.19	0.44	0.19	0.06
	9	0	0	0	0.08	0.19	0.22	0.25	0.08	0.17

Table 7: Normalized Contingency Table (column).

		Perceived strength								
		1	2	3	4	5	6	7	8	9
Entertainment	1	1.00	1.00	0	0	0.03	0	0	0.13	0.24
	2	0	0	0.14	0.04	0.02	0.02	0.02	0.13	0.12
	3	0	0	0.29	0.08	0.05	0.07	0.08	0.13	0
	4	0	0	0.14	0.08	0.14	0.09	0.08	0.06	0
	5	0	0	0.29	0.17	0.27	0.24	0.06	0	0.06
	6	0	0	0	0.25	0.17	0.15	0.12	0.06	0.12
	7	0	0	0.14	0.25	0.19	0.24	0.31	0.13	0.06
	8	0	0	0	0	0.03	0.05	0.14	0.19	0.06
	9	0	0	0	0.13	0.11	0.15	0.18	0.19	0.35

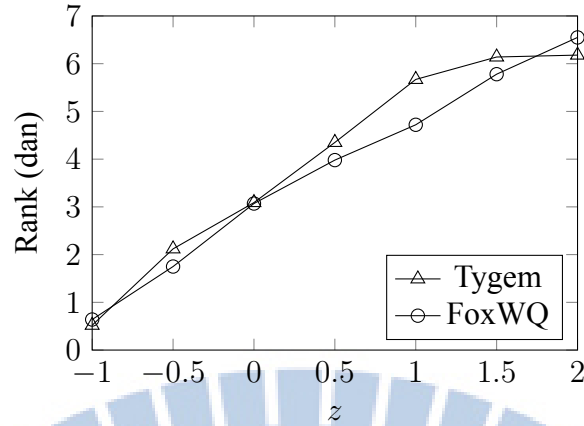


Figure 8: Dan rank strength correlation to z for the online Go platforms Tygem and FoxWQ. 0 dan corresponds to 1 kyu.

toolkits for interfacing upon request. A total of 14 differently strength-adjusted programs (7 each for dan and kyu ranks) were available to play against, for any player who wishes to do so on the two websites. On these platforms, there are options to indicate openly to the community whether an account is AI or human. To minimize irregular play from human opponents, we did not disclose this information. No surveys to human players were conducted.

Since we do not have a set of reference Go ratings for the online human opponents, we use the dan and kyu ranks on Tygem and FoxWQ as the indicator for actual program strength. The online ranking mechanism (on these platforms) is described as follows. Each new account is given an initial rank; subsequently her rank is adjusted using the most recent n games. For instance, a common setting would be to promote the player's rank by one if she has 12 wins in the most recent 20 games against players with a similar rank, or demote one rank if there were 12 losses. Once a promotion/demotion occurs, the "recent game buffer" is cleared. It is worth noting that this ranking mechanism is a dynamic system, in that each player's rank depends on the active player population's overall strength, and that it measures relative instead of absolute strength. Given a large enough game sample and a static player population, the rank should converge to a player's ability. While we cannot control for a static player population, each version of our program played at least 400 games, with an overall average of 1000 games each, of which we use the most recent 100 games for the following analyses.

Figure 8 shows that with different strength indices z , our AI program can obtain a steady distribution of strengths, ranking from 1 dan to 7 dan; z is highly correlated with dan rank

¹The player does not have a GoRating on the website. The listed rating is an estimate given by his/her peers.

Table 8: List of 17 HaiFong Professional Players.

Symbol	English Name (Chinese Name)	GoRating
P1	HSU, HAO-HUNG	3341
P2	CHEN, CHI-JUI	3223
P3	CHIEN, CHING-TING	3202
P4	LIN, LI-HSIANG	3181
P5	LAI, CHUN-FU	3058
P6	LI, WEI	3027
P7	HUANG, SHIH-YUAN	3023
P8	TSAI, CHENG-WEI	3004
P9	LU, I-CHUAN	3003
P10	HSU, CHING-EN	3000 ¹
P11	LIN, YEN-CHENG	2978
P12	YANG, TZU-HSUAN	2975
P13	LIN, CHIEH-HAN	2965
P14	NIU, SHIH-TE	2919
P15	CHANG, CHIA-HUAN	2900 ¹
P16	PAI, HSIN-HUI	2795
P17	YU, LI-CHUN	2770

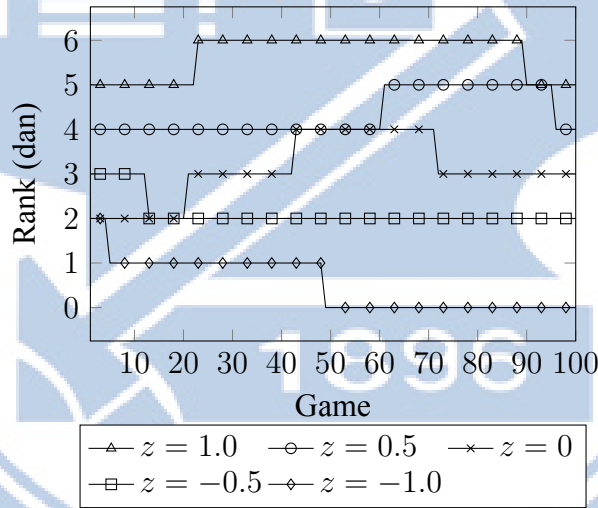


Figure 9: Tygem dan rank chart over a period of 100 games.

($R^2 = 0.9474$ for Tygem, $R^2 = 0.9931$ for FoxWQ) and a deviation of 0.5012 for Tygem and 0.6083 for FoxWQ. From Figure 9, we can see that the rank of each program stays within a range of ± 1 ranks in the most recent 100 games.

For the lower ranks from 18 kyu to 1 kyu, even with a stable playing ability, it can be difficult to maintain the same rank since beginners introduce more noise in the form of various blunders, and improve strengths more rapidly, etc. As a quick example, from a human perspective, Go can be a strenuous game to play for beginners; it is unreasonable to expect human

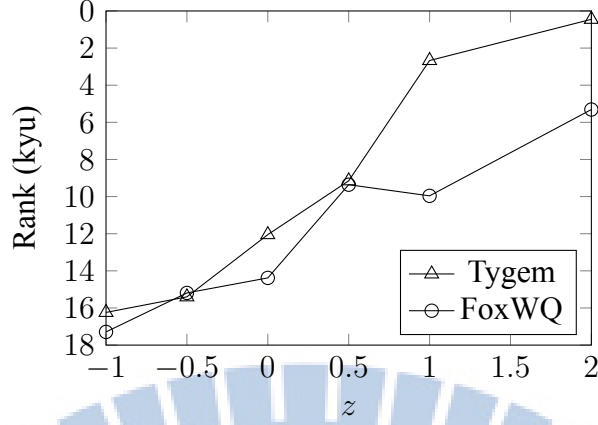


Figure 10: Kyu rank strength correlation of z on Tygem and FoxWQ. 0 kyu corresponds to 1 dan.

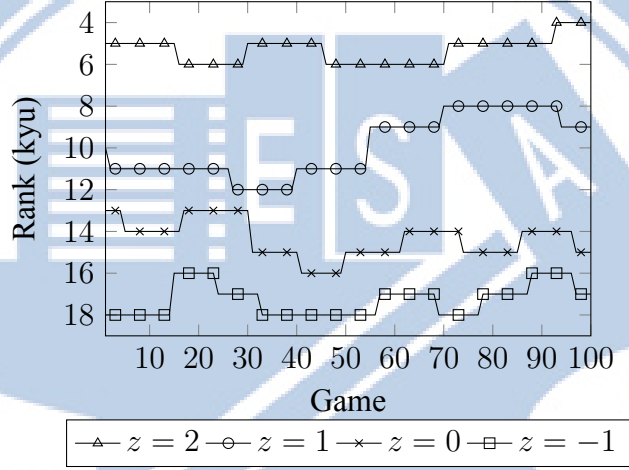


Figure 11: FoxWQ kyu rank chart over a period of 100 games.

players to commit the same level of concentration for each game at this level, especially when we did not choose the players that participated in our experiments. For these reasons, the data does not fit as well as the results for professionals or the dan ranks. Figure 10 shows the relationship between z and the programs' kyu ranks on these two websites; the correlation of z to the kyu rank is moderately high ($R^2 = 0.8985$ for Tygem, $R^2 = 0.8675$ for FoxWQ), while the deviation is much higher than the dan rank result (deviation = 1.6671 for Tygem, deviation = 1.6055 for FoxWQ). Despite the large deviation, there is still an incremental relationship between z and the average rank. We can still discriminate between different strength settings to some degree in Figure 11.

In summary, to verify that strength adjustment can be performed to obtain stable, evenly distributed programs, we performed human-program experiments in which humans ranging

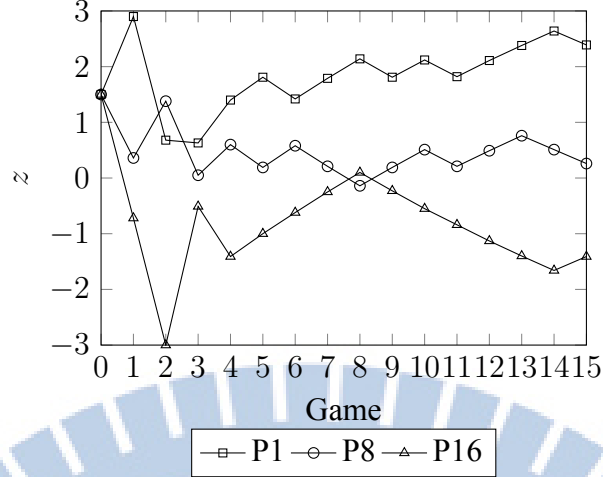


Figure 12: MDSA strength estimation of selected players.

from professional to amateur kyu rank played against our programs. The results indicate that SSA can maintain a positive strength correlation with the strength index, which enabled professional players to choose an appropriate opponent and provided game balancing, subsequently improving the entertainment value and creating more diverse game play. For the amateur levels, we showed that the strength index is highly correlated with the dan ranks, and moderately correlated with the kyu ranks.

5.2 Dynamic Strength Adjustment

We now test how effective our dynamic strength adjustment works to assess the strength of human professional players. For DSA, we directly used MDSA (as in section 4.3) to both professional and amateur players with intra-game SA for the first three games and inter-game SA for the remaining 12 game series in this experiment (15 games in total).

Each professional player was instructed to play 15 games. Different from the experiment in Figure 7, where the participants were asked to manually choose from a list of adjusted programs, in this experiment, one single version of the program with DSA was used for all participants. The participants were not informed of the adjustment mechanism.

In Figure 12, we show the MDSA estimation process for the participants P1, P8, and P16 (corresponding to the players in Table 8). The strength relation of the three players is $P1 > P8 > P16$, according to the Go Rating website Coulom (2019). The intra-game SA mechanism

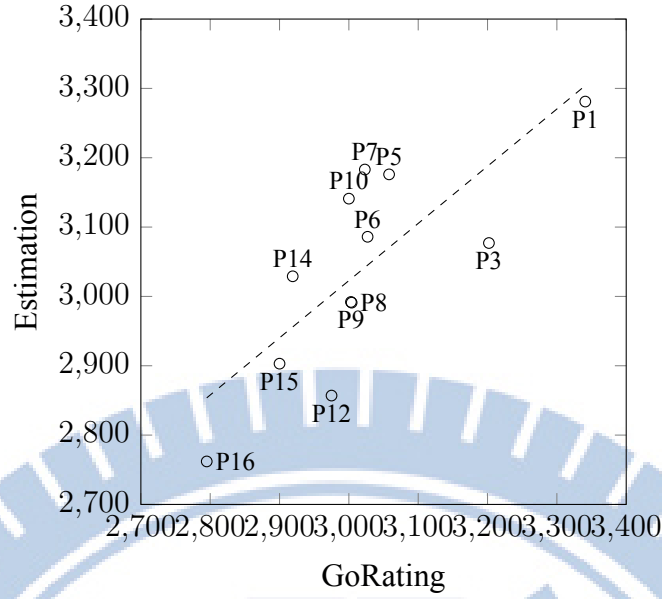


Figure 13: MDSA strength estimation of HaiFong professional players.

was used for the first three games, where we can observe a rapid changes to the value of z . At the end of the first three games, the z values appear to correspond to each player's strengths ($P1 > P8 > P16$) in order. For the remaining 12 games, inter-game SA was used; in contrast, z adjustment between the games are more stable.

The Elo rating estimation results of the 12 professional players are shown in Figure 13. Compared with SSA, the DSA mechanism estimated each player's rating with slightly less accuracy ($R^2 = 0.5986$). However, with SSA, the player had to choose the appropriate difficulty level based on their personal experience. Unlike SSA, DSA is able to adjust the program's strength automatically to fit its opponent's strength, resulting in a win rate that is close to 50% (47% ~ 57%).

Table 9 lists the z value, intra- z after intra-games and final- z after 15 games for each player. Let the inter- z -diff be intra- z minus final- z . By averaging these absolute values of inter- z -diff, we obtain the value 0.45, which we can think of as the remaining strength adjustment required by inter-game SA for the program to fit to a player's actual strength. Now, let us consider the difference between the final z and the initial z (1.5), indicated in the row of z -diff. If we average these absolute values of z -diff, we obtain the amount of adjustment required only through inter-game SA, which is about 1.08. In other words, by using intra-game SA, we are able to adjust to the player's actual strength with shorter distance, and follow up by fine-tuning with inter-game

Table 9: Intra-game SA estimation.

Label	P1	P3	P6	P5	P7	P8	P9	P10	P12	P14	P15	P16
intra-z	1.4	0.74	1.4	1.33	2.34	0.6	-0.5	1.09	-0.39	-0.02	-0.98	-1.41
final-z	2.39	0.89	0.96	1.62	1.67	0.26	0.27	1.36	-0.72	0.54	-0.38	-1.41
inter-z-diff	-0.99	-0.15	0.43	-0.29	0.67	0.34	-0.77	-0.27	0.33	-0.56	-0.6	0

Table 10: Comments of P1 (HSU, HAO-HUNG).

Game	Result	z	Comment
4	Win	1.4	I feel it's (AI) not strong in this game
5	Lose	1.81	this game is ok
6	Win	1.42	slightly weak
7	Win	1.79	moderate
8	Lose	2.14	slightly strong
9	Win	1.81	slightly weak
10	Lose	2.12	slightly strong
11	Win	1.82	weak
12	Win	2.11	slightly weak
13	Win	2.38	slightly weak
14	Lose	2.64	ok
15	Lose	2.39	strong

SA.

During the inter-game SA process, most players can detect the strength differences game by game. The strongest participant's comments are listed by game in Table 10.

Subjectively, players also reported that the programs played to balance each game. One participant reported that "the game felt close all the way to the end, even as I made constant gains". In terms of move diversity, players also reported back the following comments:

- P9: the 20th move made by CGI (W) is refreshing.
- P9: the 11th move by CGI helped broaden my perspective on the game; I think it is not a bad move.
- P12: CGI seems to play more new variations.
- P14: the winning move for CGI occurred at 101; I am rather surprised that a computer was able to come up with something like that.

VI . Discussion and Future Work

6.1 Discussion

The Elo rating in Figure 2 shows that $R_{th} = 0.05$ seems more effective to adjust strength. But, as we tried SSA on our kyu rank version Go program (CGI 1.0), the program begin to behave abnormal actions that human won't take. (E.g. keep giving up its territories). Thus, since $R_{th} = 0.1$ can be used in all cases from amateur kyu level to professional level, we suggest use 0.1 threshold ratio in computer Go.

In intra-game SA experiments, we've tried 3 different types of the MCTS win rate. First, the original MCTS win rate choosing move with most simulation count. Second, the expected MCTS win rate considering probability distribution of the softmax policy. Last, the MCTS win rate of the action we actually chose by the softmax policy. Although, using the last two types sounds more reasonable, we got similar results in our experiments. So, we just use the first one in this paper for simplicity.

6.2 Future Work

Since the strength index correlate well with logistic regression as shown in Figure 4, we can fit level using a logistic regression function. Also, in DSA mechanisms, logistic regression function can be used to help better adjustment effect.

In our proposed inter-game SA, we just use a simple adjust rule like Elo rating Elo (1978). There are some more accurate method to use, such as TrueSkill using Bayesian method or WHR Coulom (2008) using newton method to maximum posterior of ratings.

In the intra-game SA Subsection, we simply use a linear decay to the adjustment amount.

VII. Conclusion

In this paper, we propose an approach to strength adjustment for MCTS-based game-playing programs. In this approach, we follow a softmax policy Sephton et al. (2015) with a strength index z to choose moves. Most importantly, this approach uses a threshold ratio R_{th} to filter out low-quality moves i whose simulation counts in MCTS are $N_i \leq N_1 \times R_{th}$.

In practice, we applied the approach to the Go programs ELF and CGI, demonstrating the ease with which program strengths can be adjusted. The empirical results show the strength covers a range of about 830 Elo ratings with a low linear regression error of 47.95 Elo, with respect to z in the range $[-2, 2]$. To our knowledge, this result is state-of-the-art in terms of the range of strengths in Elo rating while maintaining a controllable relationship between the strength and a strength index. Another advantage is that the program is still able to play diverse moves despite its adjusted, weaker strength.

Furthermore, we present an in-depth strength analysis for the above empirical results. First, we make the hypothesis that given a position, the strength of move i is proportional to N_i^H . From this hypothesis, the strength ratings of chosen moves are shown to be at worst a fixed value, $H \times \ln R_{th}$, lower than the best move. This justifies that the move quality is under control, avoiding exceptionally bad moves. In addition, the analysis also shows that the derived strengths are also close to the empirical strengths with regression error 40.45 Elo, and to a logistic function with regression error 10.51 Elo.

With the ease of strength adjustment using z , we introduce three methods to adjust strength dynamically, including inter-game, intra-game and mixed dynamic strength adjustment. The experiment results show that these methods are able to predict the opponents' expected strengths, though the variances can be high.

Finally, we test our methods in real-world cases against professionals, dan rank amateurs

and kyu rank amateurs. The players are able to subjectively judge the strength of the programs corresponding to different strength indices. From player surveys, we conclude that professionals tend to feel more entertained when the program is slightly above their strength. When our strength adjusted program is made available online, it can play consistently at evenly-distributed ranks according to each program’s preset strength index. As a player performance assessment tool, our program can predict opponent strength accurately in 15 games.

We are currently in the process of applying our strength adjustment mechanism into a so-called *lifelong learning system* for a variety of games. As a user grows in playing ability, the lifelong learning system is able to keep up with the user and provide appropriate opponents at all levels of play. We have demonstrated in this paper that our method can cover all ranks on Go websites, and even for professional players. The lifelong learning system using our strength adjustment method is therefore suitable for long-term learning, training, and entertainment.

In terms of future work, the AlphaZero algorithm has also been successfully applied to other games such as chess and shogi, also achieving super-human level strength Silver et al. (2017a). With our approach, we expect to be able to provide a wide range of strength levels for each of these games. We expect our approach to not only impact the Go community, but also the games community at large.

Reference

- Arno, H. and Morten, P. (2015). Sensei's library.
- Cauwet, M.-L., Teytaud, O., Cazenave, T., Saffidine, A., Liang, H.-M., Yen, S.-J., Lin, H.-H., and Wu, I.-C. (2015). Depth, balancing, and limits of the Elo model. In *2015 IEEE Conference on Computational Intelligence and Games (CIG)*, pages 376–382. IEEE.
- Computer Shogi Association (2019). Results of the 27th world computer shogi championship.
- Coulom, R. (2007). Computing Elo ratings of move patterns in the game of Go. *ICGA Journal*, 30(4):198–208.
- Coulom, R. (2008). Whole-history rating: A Bayesian rating system for players of time-varying strength. In *International Conference on Computers and Games*, pages 113–124. Springer.
- Coulom, R. (2019). The website of Go ratings.
- Demediuk, S., Tamassia, M., Raffe, W. L., Zambetta, F., Li, X., and Mueller, F. (2017). Monte Carlo tree search based algorithms for dynamic difficulty adjustment. In *2017 IEEE Conference on Computational Intelligence and Games (CIG)*, pages 53–59. IEEE.
- Elo, A. E. (1978). *The rating of chess players, past and present*. Arco Pub.
- HaiFong Go Association (2018). The homepage of HaiFong Go association.
- Hosmer Jr, D. W., Lemeshow, S., and Sturdivant, R. X. (2013). *Applied logistic regression*, volume 398. John Wiley & Sons.
- Hunicke, R. and Chapman, V. (2004). AI for dynamic difficulty adjustment in games. In *Proceedings of the AAAI-04 Workshop on Challenges in Game Artificial Intelligence*.

- Hunter, D. R. et al. (2004). MM algorithms for generalized Bradley-Terry models. *The annals of statistics*, 32(1):384–406.
- Kocsis, L. and Szepesvári, C. (2006). Bandit based Monte-Carlo planning. In *European conference on machine learning*, pages 282–293. Springer.
- Pascutto, G.-C. (2018). Leela-Zero GitHub repository.
- Paulsen, P. and Fürnkranz, J. (2010). A moderately successful attempt to train chess evaluation functions of different strengths. In *Proceedings of the ICML-10 Workshop on Machine Learning and Games, Haifa, Israel*.
- Romstad, T., Costalba, M., Kiiski, J., et al. (2019). Stockfish: a strong open source chess engine.
- Sephton, N., Cowling, P. I., and Slaven, N. H. (2015). An experimental study of action selection mechanisms to create an entertaining opponent. In *2015 IEEE Conference on Computational Intelligence and Games (CIG)*, pages 122–129. IEEE.
- Shih, C.-C., Liu, A.-J., Wu, I.-C., et al. (2018). 2017 “CITIC Securities Cup” – the 1st world AI Go open. *ICGA Journal*, 40(4):363–368.
- Silva, M. P., do Nascimento Silva, V., and Chaimowicz, L. (2017). Dynamic difficulty adjustment on MOBA games. *Entertainment Computing*, 18:103–123.
- Silver, D., Huang, A., Maddison, C. J., Guez, A., Sifre, L., Van Den Driessche, G., Schrittwieser, J., Antonoglou, I., Panneershelvam, V., Lanctot, M., et al. (2016). Mastering the game of Go with deep neural networks and tree search. *nature*, 529(7587):484–489.
- Silver, D., Hubert, T., Schrittwieser, J., Antonoglou, I., Lai, M., Guez, A., Lanctot, M., Sifre, L., Kumaran, D., Graepel, T., et al. (2017a). Mastering chess and shogi by self-play with a general reinforcement learning algorithm. *arXiv preprint arXiv:1712.01815*.
- Silver, D., Schrittwieser, J., Simonyan, K., Antonoglou, I., Huang, A., Guez, A., Hubert, T., Baker, L., Lai, M., Bolton, A., et al. (2017b). Mastering the game of Go without human knowledge. *Nature*, 550(7676):354.
- Tencent (2019). The website of Fox Weiqi.

Tencent AI Lab (2018). Fine art wikipedia.

Tongyang Online (2019). The homepage of Tygem.

Wu, I.-C., Wu, T.-R., Liu, A.-J., Guei, H., and Wei, T. (2019). On Strength Adjustment for MCTS-Based Programs. In *Thirty-Third AAAI Conference on Artificial Intelligence*.

Wu, T.-R., Wu, I.-C., Chen, G.-W., Wei, T.-h., Wu, H.-C., Lai, T.-Y., and Lan, L.-C. (2018). Multilabeled value networks for computer Go. *IEEE Transactions on Games*, 10(4):378–389.

Yuandong Tian, Jerry Ma*, Qucheng Gong*, Shubho Sengupta*, Zhuoyuan Chen, James Pinkerton, and C. Lawrence Zitnick (2019). ELF OpenGo: An analysis and open reimplementation of AlphaZero. *CoRR*, abs/1902.04522.

